Demystifying Docker

A Comprehensive Guide to Managing Containerized Applications

Understanding Docker and Related Concepts

In this document, we will delve into the essential components of Docker and related concepts that are pivotal for managing containerized applications. Each section offers a comprehensive breakdown of individual topics, providing a foundational understanding for those new to Docker or looking to deepen their knowledge.

1. Basic Docker Commands

Docker commands are the building blocks for interacting with Docker containers and images. Here's a look at some fundamental commands:

- **Pull an image:**
  docker pull postgres
  This command fetches the specified image from Docker Hub. If no tag is specified, Docker defaults to the latest version.
- **List containers:**
  - docker ps: Displays only running containers.
  - docker ps -a: Lists all containers, including those that have stopped.
- **List images:**
  docker images: Shows all images that are available locally.
- **Stop container:**
  docker stop <container_id>: Halts the specified container.
- **Remove stopped containers:**
  docker container prune: Cleans up all stopped containers to free up resources.

2. Running PostgreSQL with Environment Variables

Environment variables are crucial for configuring containerized applications. In the case of PostgreSQL:

- **Example syntax:**

- docker run -d \
  --name my-postgres \
  -e POSTGRES_PASSWORD=123456789 \
  postgres
- The -e flag sets environment variables. PostgreSQL requires a password for authentication, which is specified here.

3. Volumes (VERY IMPORTANT)

Volumes play a critical role in data persistence:

- **Why is data NOT deleted?**
  Due to the use of anonymous volumes, data persists even after a container is removed.
- **Internal workings:**
  The PostgreSQL image designates a volume path (/var/lib/postgresql/data), and Docker automatically creates a volume if not specified.
- **Key Concept:**
  - Containers are temporary, meaning they can be removed without affecting the data.
  - Volumes are persistent, ensuring data survives container removal.
- **Deleting volumes:**
  docker volume prune: Removes unused volumes to reclaim space.

4. MongoDB Default Port

MongoDB typically listens on port **27017** by default. Knowing default ports is essential for network configurations and troubleshooting.

5. Docker Networking

Understanding Docker networking enhances container communication:

- **Default Network:**
  Docker uses a bridge network by default, allowing containers on the same host to communicate.
- **User-Defined Bridge Network:**
  - **Create network:** docker network create my-private-net
  - **Run containers:**
  - docker run -d --name app --network my-private-net nginx
    docker run -d --name db --network my-private-net redis
  - **Benefits:**
    - Containers communicate using names instead of IP addresses.
    - Built-in DNS support for ease of use.
    - Increased security within the network.

6. NAT (Internet Access)

NAT, or Network Address Translation, allows containers to access the internet:

- **Flow:**
  - A container sends a request.
  - The request goes to the Docker gateway (e.g., 172.x.x.1).
  - The host machine processes the request using NAT.
  - The internet response returns to the container.

7. IP Assignment in Docker Network

When multiple containers exist, IP addresses are assigned sequentially:

- **Example:**
  - **Gateway:** 172.x.x.1
  - **Container 1:** 172.x.x.2
  - **Container 2:** 172.x.x.3

8. Security Risks

Security is paramount in any networked environment:

- **Port Exposure:**
  The command -p 80:80 maps the container port to the host, exposing it to the internet.
- **Privileged Mode:**
  The --privileged flag grants full access to the host, which can be dangerous if misused.

9. Reverse Proxy vs Forward Proxy

Understanding proxies helps in configuring secure and efficient networks:

- **Forward Proxy:**
  Protects the client by acting as an intermediary for requests, commonly used in corporate firewalls.
- **Reverse Proxy:**
  Protects the server and manages traffic, often implemented with Nginx.
  Flow:
  - User requests → reverse proxy → internal container → response back to user.

10. Ping (Networking)

Ping is a basic network utility that uses the Internet Control Message Protocol to test connectivity:

- **Process:**
  - Sends an "echo request" packet to a target.
  - Receives an "echo reply" if the target is reachable.

11. Docker Swarm (Cluster)

Docker Swarm is a native clustering and orchestration tool for managing Docker:

- **What is Swarm?**
  A collection of Docker engines working together as one system.
- **Initialize Swarm:**
  docker swarm init --advertise-addr <IP>
- **Join another node:**
  docker swarm join --token <token> <manager-ip>:2377

12. Overlay Network (Swarm)

An overlay network allows containers to communicate across different hosts in a Swarm:

```
docker network create \
  --driver overlay \
  --attachable my_swarm_network
```

13. Services in Swarm

Services define a task or a set of tasks to be executed in the Swarm:

```
docker service create \
  --name web \
  --replicas 2 \
  --publish published=8080,target=80 \
  nginx
```

14. Raft Consensus Algorithm

The Raft consensus algorithm ensures consistency across a distributed system:

- **Consensus:**
  Agreement among nodes on a decision.
- **Quorum Formula:**
  (N / 2) + 1 determines the majority needed for consensus.
- **Importance:**
  Prevents "split-brain" scenarios and ensures data consistency.

15. Swarm Failure Case

In the event of node failures in a Swarm:

- **Example:**
  If there are 3 managers and 2 are lost, there is no quorum.
- **Result:**
  - Running containers continue to function.
  - New changes are blocked until quorum is restored.

16. Key Concepts Summary

- **Containers** are temporary entities.
- **Volumes** ensure data persistence.
- **Bridge networks** operate within the same host.
- **Overlay networks** span multiple hosts.
- **Reverse proxies** offer enhanced security and request routing.
- **Swarm** is used for orchestrating container clusters.
- **Raft** is the decision-making mechanism that ensures consistency.

If you would like further customization, such as converting this into exam notes, setting up a real-world project, or creating interview questions, feel free to ask!